

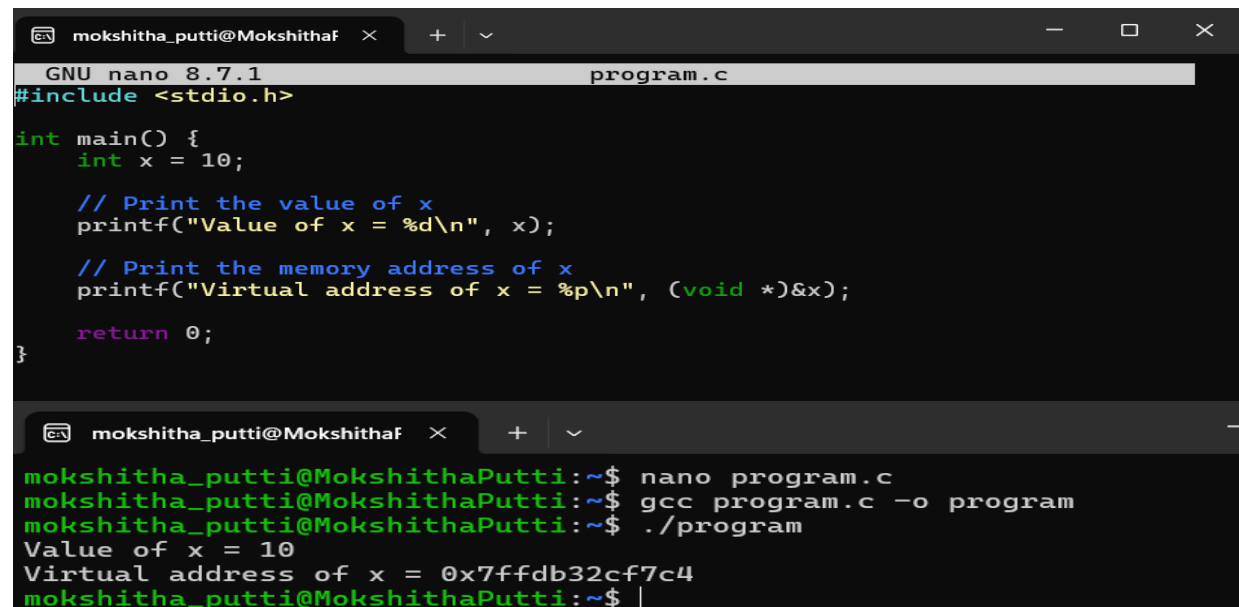
Virtual address 1

```
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```



```
mokshitha_putti@MokshithaF x + v - □ ×
GNU nano 8.7.1 program.c
#include <stdio.h>

int main() {
    int x = 10;

    // Print the value of x
    printf("Value of x = %d\n", x);

    // Print the memory address of x
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}

mokshitha_putti@MokshithaPutti:~$ nano program.c
mokshitha_putti@MokshithaPutti:~$ gcc program.c -o program
mokshitha_putti@MokshithaPutti:~$ ./program
Value of x = 10
Virtual address of x = 0x7ffdb32cf7c4
mokshitha_putti@MokshithaPutti:~$ |
```

2 memory segment

```
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address      : %p\n", (void *)main);
    printf("Data address       : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address       : %p\n", (void *)heap);
    printf("Stack address      : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

```

GNU nano 8.7.1 demandpage.c
#include <stdio.h>
#include <stdlib.h>

int global = 10;           // Stored in Data segment
int bss_variable;          // Stored in BSS segment (uninitialized global)

int main() {
    int local = 20;         // Stored in Stack

    int *heap = malloc(sizeof(int)); // Stored in Heap
    *heap = 30;

    // Print addresses from different segments
    printf("Code address      : %p\n", (void *)main);
    printf("Data address       : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address         : %p\n", (void *)heap);
    printf("Stack address        : %p\n", (void *)&local);

    free(heap); // Always free heap memory
}

```

[Read 23 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Loca
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go

```

mokshitha_putti@MokshithaPutti:~$ nano program.c
mokshitha_putti@MokshithaPutti:~$ nano demandpage.c
mokshitha_putti@MokshithaPutti:~$ gcc demandpage.c -o demandpage
mokshitha_putti@MokshithaPutti:~$ ./demandpage
Code address      : 0x5a974e7f11a9
Data address       : 0x5a974e7f4010
BSS address        : 0x5a974e7f4018
Heap address       : 0x5a975399b010
Stack address      : 0x7ffc17303d7c

```

Demandpagging 3

```

#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

```
GNU nano 8.7.1 demandpagging.c
#include <stdio.h>

int main() {
    int page_table[4] = {1, -1, 3, -1}; // -1 means page not loaded
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

```
mokshitha_putti@MokshithaPutti:~$ nano demandpage.c
mokshitha_putti@MokshithaPutti:~$ nano demandpagging.c
mokshitha_putti@MokshithaPutti:~$ nano demandpagging.c
mokshitha_putti@MokshithaPutti:~$ gcc demandpagging.c -o demandpagging
mokshitha_putti@MokshithaPutti:~$ ./demandpagging
Enter page number (0-3): 0
Page is already in memory.
Physical frame = 1
mokshitha_putti@MokshithaPutti:~$
```

=====

=====

address_translation 4

```

#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address   = %d\n", virtual_address);
    printf("Page Number       = %d\n", page);
    printf("Offset               = %d\n", offset);
    printf("Frame Number          = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}

```

```

GNU nano 8.7.1 address_translation.c
#include <stdio.h>

int main() {
    int page_table[] = {3, 5, 1, 7};    // Example page table
    int page_size = 100;                // Size of each page in bytes

    int virtual_address = 250;          // Example virtual address

    // Calculate page number and offset
    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    // Find frame number from page table
    int frame = page_table[page];

    // Calculate physical address
    int physical_address = frame * page_size + offset;

    // Print results
    printf("Virtual Address   = %d\n", virtual_address);
    [ Read 27 lines ]

```

```

mokshitha_putti@MokshithaPutti:~$ nano address_translation.c
mokshitha_putti@MokshithaPutti:~$ gcc address_translation.c -o address_translation
mokshitha_putti@MokshithaPutti:~$ ./address_translation
Virtual Address   = 250
Page Number       = 2
Offset           = 50
Frame Number      = 1
Physical Address  = 150
mokshitha_putti@MokshithaPutti:~$ |

```

=====

=====

Malloc 5

```
mokshitha_putti@MokshithaPutti x + v
GNU nano 8.7.1 malock.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    // Allocate memory for 5 integers
    p = malloc(5 * sizeof(int));

    // Check if allocation succeeded
    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Assign values to allocated memory
    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10; // 10, 20, 30, 40, 50
    }

    [ Read 31 lines ]
    ^G Help    ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

mokshitha_putti@MokshithaPutti:~$ nano malock.c
mokshitha_putti@MokshithaPutti:~$ gcc malock.c -o malock
mokshitha_putti@MokshithaPutti:~$ ./malock
10 20 30 40 50
=====
=====
```

Segmentation memory I used name as null _pointer.c 6

```
#include <stdio.h>

int main() {
    int *p = NULL;
    printf("Before accessing memory\n");
    *p = 100;
    printf("After accessing memory\n");
    return 0;
}
```

```
mokshitha_putti@MokshithaPutti x + v
GNU nano 8.7.1 null_pointer.c
#include <stdio.h>

int main() {
    int *p = NULL;
    printf("Before accessing memory\n");
    *p = 100;    // This will cause a segmentation fault
    printf("After accessing memory\n");
    return 0;
}

mokshitha_putti@MokshithaPutti:~$ nano malock.c
mokshitha_putti@MokshithaPutti:~$ nano null_pointer.c
mokshitha_putti@MokshithaPutti:~$ gcc null_pointer.c -o null_pointer
mokshitha_putti@MokshithaPutti:~$ ./null_pointer
Before accessing memory
Segmentation fault ./null_pointer
```